

WHAT IS A COMPUTER?

A TOOL . . . A MACHINE . . .
THE LOVE OF MY LIFE...

The ultimate tool in its ability to be reconfigured for different purposes.

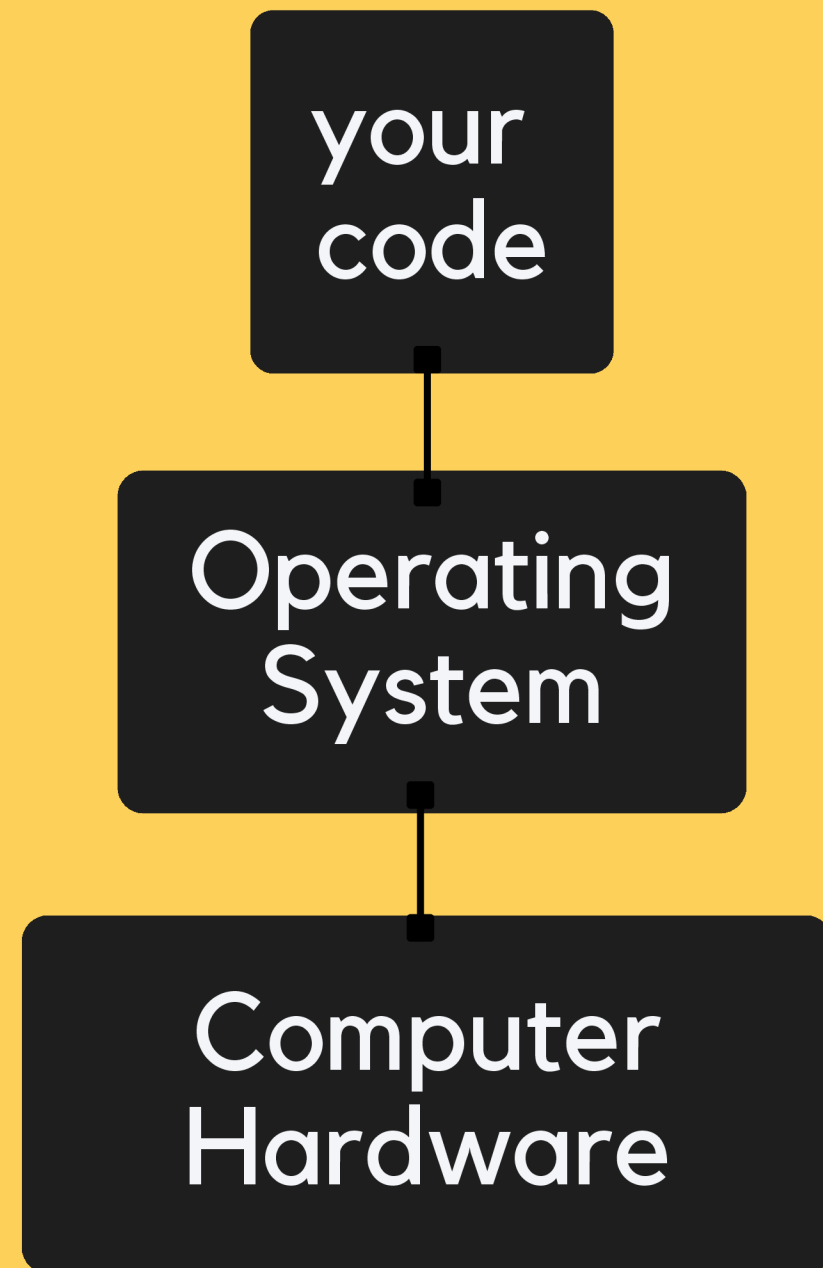
The key elements:

- A processor to execute commands
- Memory to store information

WHAT IS PROGRAMMING?

- Providing a computer with specific instructions to solve various problems
 - Using specific languages to write those instructions (code)
- At the core of it - problem solving!
 - You may go through many iterations before you get it right
 - mistakes are good!

WHAT IS AN OPERATING SYSTEM?



- An Operating System is the interface between the user and the computer hardware
- Operating Systems:
 - Execute user programs and make solving problems easier
 - Make the computer system convenient to use
- Basically, an Operating System sits between our code and the computer, providing essential services

WHAT IS LINUX?

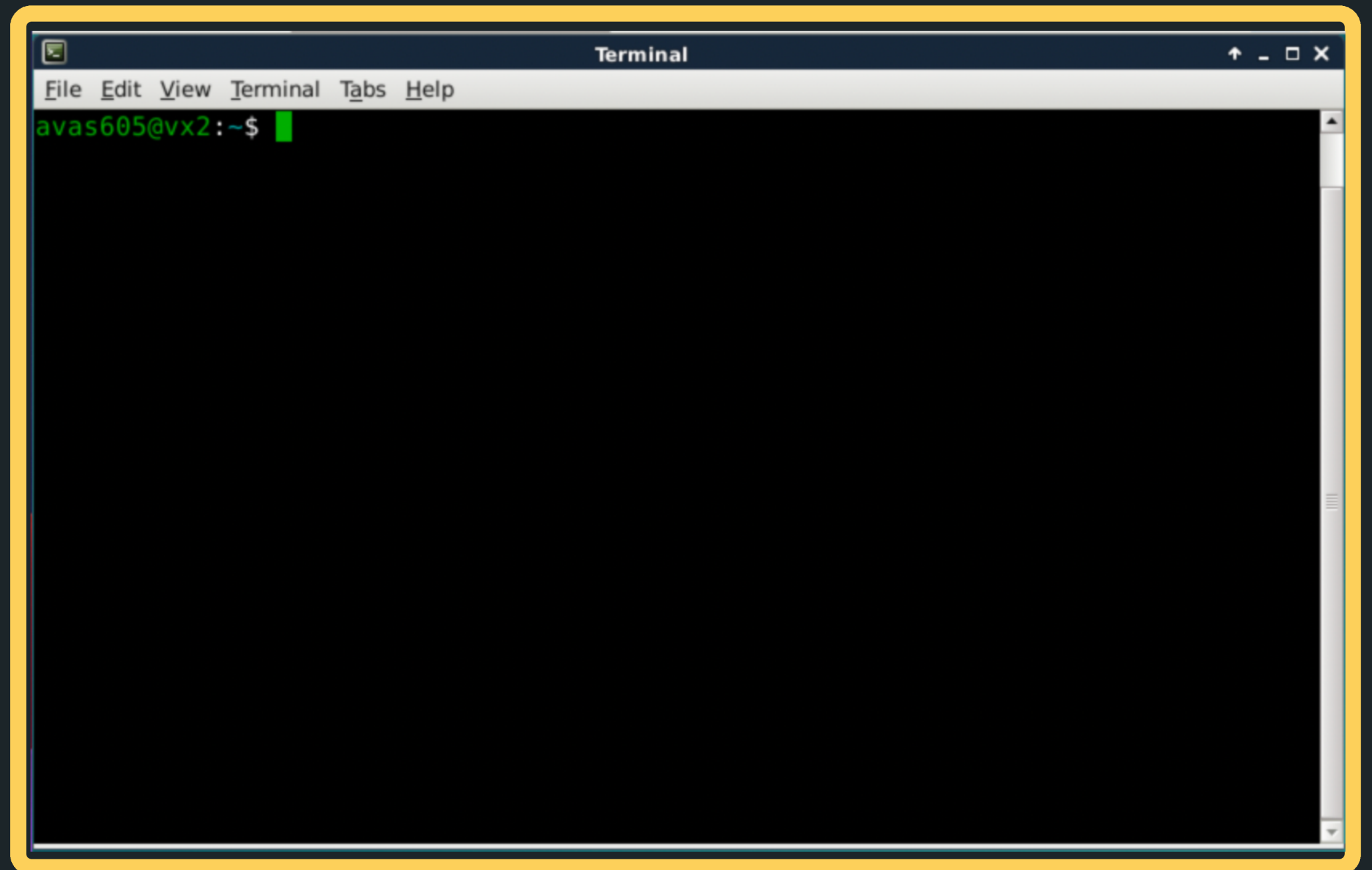


- Linux is a Unix-based operating system:
 - Open source
 - More reliable
 - Lightweight
 - Faster, and
 - More secure

TERMINAL

A GRAPHICAL APPLICATION THAT READS/DISPLAYS INFORMATION

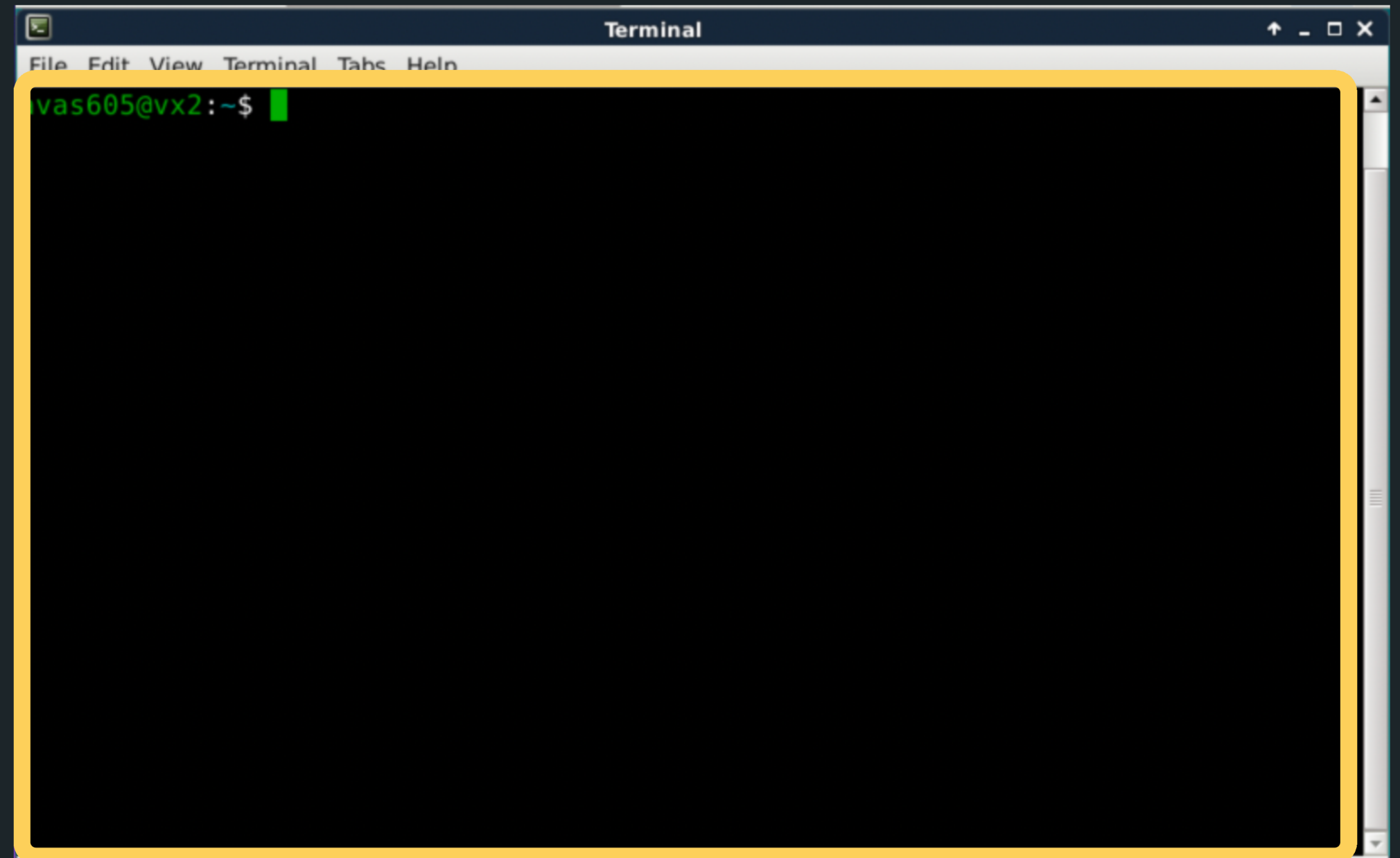
- Terminal (command line driven) allow us to send simple text commands to our shell. It handles things like user input, displaying shell output.



SHELL

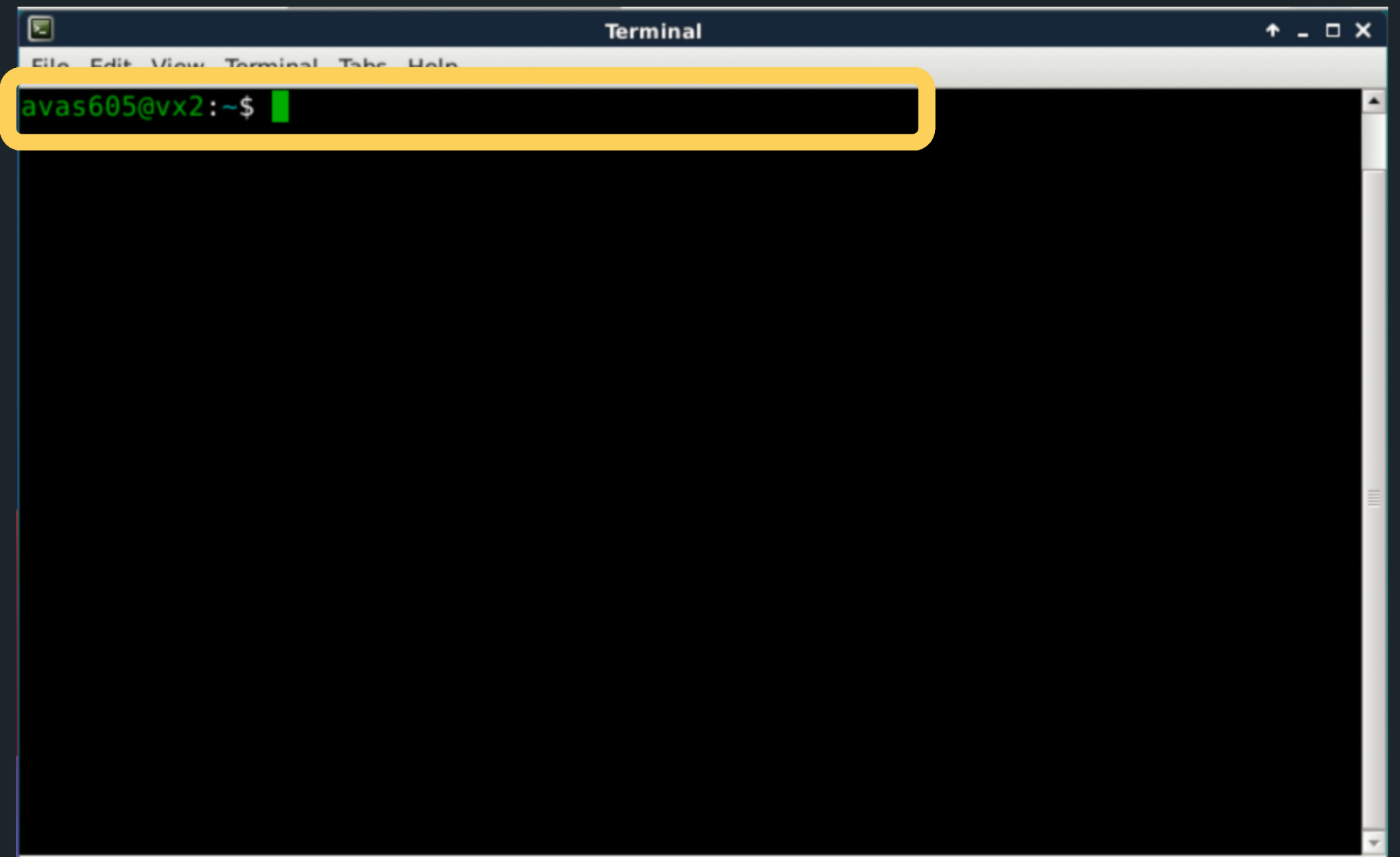
PRIMARY INTERFACE WITH THE COMPUTER

- The shell, (bash, zsh) is a program that executes commands, and has its own syntax. It returns output which the terminal can display, or can launch other applications



PROMPT

- The prompt is controlled by the shell, and is the line of text which displays some information



SOME IMPORTANT TERMINAL COMMANDS

- Lists all the files in the current directory:
ls
- Makes a new directory called `directoryName`:
mkdir `directoryName`
- Changes the current directory to `directoryName`:
cd `directoryName`
- Moves up one level of directories (one folder level):
cd ..
- Tells you where you are in the directory structure at the moment:
pwd

COMMAND LINE AND FILE OPERATIONS

File operations on the command line

- Copy a file from the source to the destination

cp source destination

- Move a file from the source to the destination (can also be used to rename)

mv source destination

- Remove a file (delete)

rm filename

The -r tag can be added to cp or rm commands to recursively go through a directory and perform the command on all the files

cp -r COMP1511 COMP1511_backup

(will copy all files from my COMP1511 directory to my COMP1511_backup directory)

USING CSE'S COMPUTING RESOURCES

Our labs are running Linux with the basic tools necessary to get started

You will definitely want to get your own computer ready to code with:

- VLAB allows you to remotely use CSE's resources - instructions on setting this up available in the first laboratory
- There are other more advanced options that we can help you with also - check the Home Computing site or the guides on our course website

WHAT THE BASICS LOOK LIKE

For COMP1511 we need:

- A development environment (we will use a minimal version of VSCode)
 - Run **1511 setup** to get everything ready (you will do this in your first Lab)
- A compiler (we use dcc)
 - A translator that takes our formal human readable C and turns it into the actual machine readable program
 - The result of the compiler is a program we can "run"
- You can use VLAB to access CSE's editor and compiler

PROGRAMMING IN C

**PROGRAMMING IS
LIKE TALKING TO
YOUR COMPUTER**

- We need a shared language to be able to have this conversation
- We'll be looking at one particular language, C and learning how to write it. C is:
 - A clear language with defined rules so that nothing we write in it is ambiguous
 - Many modern programming languages are based on C
 - A good starting point for learning how to control a computer from its roots

LET'S C SOME C

SORRY CAN'T HELP MYSELF!

```
1 // A demo program showing output in C
2 // Welcome to COMP1511 :)
3 // Buckle, it is going to be a wild ride with
4 // a steep learning curve that we will hit in
5 // about Week 3!
6
7 #include <stdio.h>
8
9 int main(void){
10     printf("Welcome to COMP1511!\n");
11     return 0;
12 }
```

BREAKING IT DOWN INTO PARTS

HEADER (LINES 1-5)

```
1 // A demo program showing output in C
2 // Welcome to COMP1511 :)
3 // Buckle, it is going to be a wild ride with
4 // a steep learning curve that we will hit in
5 // about Week 3!
6
7 #include <stdio.h>
8
9 int main(void){
10     printf("Welcome to COMP1511!\n");
11     return 0;
12 }
```

- Words for humans
- Half our code is for the machine, the other half is for humans! (roughly)
- We put “comments” in to describe to our future selves or our colleagues what we intended for this code
- `//` in front of a line makes it a comment
- If we use `/*` and `*/` everything between them will be comments
- The compiler will ignore comments, so they don't have to be proper code

BREAKING IT DOWN INTO PARTS

#INCLUDE IS A SPECIAL TAG FOR OUR COMPILER (LINE 7)

```
1 // A demo program showing output in C
2 // Welcome to COMP1511 :)
3 // Buckle, it is going to be a wild ride with
4 // a steep learning curve that we will hit in
5 // about Week 3!
6
7 #include <stdio.h>
8
9 int main(void){
10     printf("Welcome to COMP1511!\n");
11     return 0;
12 }
```

- It asks the compiler to grab another file of code and add it to ours
- In this case, it's the Standard Input Output Library, allowing us to make text appear on the screen (as well as other things)
- Almost every C program you will write in this course will have this line

BREAKING IT DOWN INTO PARTS

THE "MAIN" FUNCTION (LINES 9-12)

```
1 // A demo program showing output in C
2 // Welcome to COMP1511 :)
3 // Buckle, it is going to be a wild ride with
4 // a steep learning curve that we will hit in
5 // about Week 3!
6
7 #include <stdio.h>
8
9 int main(void){
10     printf("Welcome to COMP1511!\n");
11     return 0;
12 }
```

- A function is a block of code that is a set of instructions that returns something
- Our computer will run this code line by line, executing our instructions
- The first line has details that we'll cover in later lectures
 - `int` is the output (return) type - this stands for integer, which is a whole number
 - `main` is the name of the function
 - `(void)` means that this function doesn't take any input

BREAKING IT DOWN INTO PARTS

THE "MAIN" FUNCTION

```
1 // A demo program showing output in C
2 // Welcome to COMP1511 :)
3 // Buckle, it is going to be a wild ride with
4 // a steep learning curve that we will hit in
5 // about Week 3!
6
7 #include <stdio.h>
8
9 int main(void){
10     printf("Welcome to COMP1511!\n");
11     return 0;
12 }
```

- Between the { and } are a set of program instructions
{ }
- `printf()` makes text appear on the screen. It is actually another function from `stdio.h` which we included.
`printf("Hey!\n");`
- `return` is a C keyword that says we are now delivering the output of the function. A main that returns 0 is signifying a correct outcome of the program
`return 0;`